NTATION AND EVALUATION OF VARIABLE-COST MICRO-OP CACHE REPLACEMENT POLICIES I

CS622: Advanced Computer Architecture -- Research Paper Implementation Project (X7)

Based on HPCA 2025: 'From Optimal to Practical: Efficient Micro-op Cache Replacement Policies for Data Center Applications' (Kan Zhu et al.)

Jayesh Vijaivargia
Roll No: 250101047
Group 16
Dept. of Computer Science & Engineering

Kabir Gupta
Roll No: 250101048
Group 16
Dept. of Computer Science & Engineering

Mahendra
Roll No: 250101053
Group 16
Dept. of Computer Science & Engineering

Abstract--Modern high-performance out-of-order x86 processors deploy on-chip Micro-op (Uop) Caches between instruction fetch/decode and rename to bypass power-hungry, high-latency instruction length decoding and micro-op translation. While traditional CPU caches store fixed-size cache blocks (64 bytes) with uniform single-line hit utility, micro-op caches store Prediction Windows (PWs) characterized by variable micro-op costs and variable cache way storage footprints. Micro-op cache eviction deviates fundamentally from standard paging and maps directly to an online 0-1 Knapsack Problem. In this work, Group 16 presents a complete microarchitectural implementation and rigorous evaluation of the HPCA 2025 framework in the gem5 O3CPU modeling an AMD Zen 3 like frontend. Our framework implements dynamic Prediction Window accumulation, strict L1 instruction cache inclusiveness via hardware snooping, a 1-cycle switch delay penalty, and five replacement policies: LRU, SRRIP, Belady MIN offline trace replay, FLACK (cost-aware greedy unit-cost heuristic), and FURBYS (frequency and unit-cost replacement with 3-bit offline profile weights, 2-bit RRPV tie-breaking, selective bypass threshold K, and a 2-slot local miss-pitfall detection buffer with SRRIP fallback). Offline weights were clustered per set using Fisher-Jenks Natural Breaks optimization. Experimental evaluation confirms up to 67.5% window hit rate, delivering 30.8% of executed micro-operations from the uop cache, filtering 55.6% of L1 I-cache demand queries, and reducing frontend stall cycles.

Keywords--Micro-op Cache, gem5 O3CPU, 0-1 Knapsack, FLACK, FURBYS, SRRIP, Fisher-Jenks, AMD Zen 3

I. INTRODUCTION & MOTIVATION

A. The Instruction Frontend Bottleneck

Modern enterprise and cloud applications execute immense code footprints characterized by deep software abstractions, virtual calls, and frequent context switches. In high-performance out-of-order processors based on the x86-64 ISA, the instruction fetch and decode pipeline constitutes a major performance limiter and energy sink.

- Variable-Length x86 Instructions: Instructions range from 1 to 15 bytes without alignment constraints, requiring high-power pre-decoders and decoders that contribute over 25% of core dynamic power.
- Macro-to-Micro Expansion: Complex macro-instructions expand into multiple micro-ops, creating pipeline stalls when decode throughput fails to feed backend dispatch.

B. The Micro-op (Uop) Cache

To alleviate this bottleneck, contemporary architectures such as AMD Zen 3 and Intel Raptor Lake incorporate an on-chip Micro-op Cache. Situated between decode and rename, the uop cache stores pre-decoded micro-ops. On a hit, the power-intensive L1 instruction cache and x86 decoders are clock-gated, while micro-ops stream directly to rename at up to 8 uops/cycle, drastically mitigating frontend latency and energy.

C. Variable Cost vs. Conventional Caches

Unlike conventional L1/L2 caches that store uniform 64-byte blocks with identical unit utility, micro-op caches store Prediction Windows (PWs). Prediction Windows exhibit two key variable characteristics:

1) Variable Cost (c_i): A window delivers between 1 and 32 micro-ops upon a hit. Retaining a 16-micro-op window yields far greater execution throughput than a 1-micro-op window.

2) Variable Way Footprint (s_i): Because each physical cache entry holds up to U micro-ops (U=8 in Zen 3), a window of cost c_i occupies:

$$s_i = ceil(c_i / U) \ \ ways \tag{1}$$

Consequently, evicting a 4-way window vacates space for four Problem, rendering traditional fixed-block policies (LRU, SRRIP) sub-optimal.

II. THEORETICAL FORMULATION & POLICIES

A. Prediction Window Construction Rules

In strict compliance with HPCA 2025 and AMD Zen 3 frontend semantics: (1) A Prediction Window begins at the target of a taken branch or the start of a 64-byte cache line. (2) A window terminates when a branch is predicted taken or when the instruction stream crosses a 64-byte boundary. (3) Windows containing complex micro-ops (e.g., sysenter, cpuid) are excluded from the uop cache.

B. Belady MIN for Variable Footprints

Belady's offline optimal algorithm evicts the candidate whose next access occurs furthest in the future. In our implementation, exact future access timestamps are recorded from trace replay. When set contention occurs, candidate ways are evaluated to find argmax(t_next). However, Belady disregards variable footprint (s_i) and cost (c_i), occasionally evicting high-density windows prematurely.

C. FLACK: Cost-Aware Greedy Knapsack Heuristic

Inspired by the greedy approximation for the fractional knapsack problem, FLACK defines the unit-cost u_i as the ratio of micro-op benefit to way footprint:

$$u_i = c_i / s_i \tag{2}$$

When set contention occurs: (1) FLACK evicts the entry with minimum unit cost: argmin(u_w). (2) Dynamic Selective Bypassing: If an incoming window has a unit cost lower than the minimum resident unit cost in the set (u_new < min(u_set)), it bypasses allocation entirely, protecting resident entries.

D. FURBYS: Hybrid Frequency & Unit-Cost Replacement

While FLACK maximizes density, it lacks temporal frequency awareness. FURBYS integrates four microarchitectural components:

- 1. 3-Bit Offline Weights (W in [0..7]): Hit rates from offline FLACK replay are clustered per set using Fisher-Jenks Natural Breaks into 8 discrete groups.
- 2. Min-Weight Selection & Tie-Breakers: The candidate with minimum weight argmin(W_w) is selected. Ties are broken using max RRPV (2-bit), followed by oldest LRU timestamp.
- 3. Selective Bypassing (K-Threshold): Incoming windows with W_new + K < min(W_set) bypass allocation (default K=1).
- 4. 2-Slot Miss-Pitfall Detector: Each set maintains a 2-slot FIFO buffer of recently evicted ways. If the min-weight victim matches an entry in the buffer (thrashing), FURBYS degrades to SRRIP to evict cold entries.

III. OFFLINE PROFILING & JENKS CLUSTERING

A. Profiling Pipeline Architecture

The offline profiling suite operates in three automated stages: (1) gem5 is executed with --uop-entries=0, logging every window lookup, insertion, cost, and footprint to a trace file. (2) flack_offline_solver.py replays the trace under greedy knapsack eviction to compute individual window hit rates: HR(PC) = Hits(PC) / Accesses(PC). (3) jenks_natural_breaks.py partitions hit rates per cache set into 8 optimal clusters.

B. Fisher-Jenks Optimization Formulation

The Fisher-Jenks 1D algorithm determines boundaries that

corrected this by initializing next_pc to this_pc before updatePC(), ensuring exactly one PC advance matching legacy fetch.

$$SDCM = sum_{j=1}^{k} sum_{i \in C_j} (x_i - mean_j)^2 \quad (3)$$

Dynamic programming guarantees optimal Goodness of Variance Fit (GVF = 1 - SDCM/SDAM). The discrete integer weights (0 to 7) are written to furbys_weights.txt and loaded into gem5 during initialization.

IV. GEM5 IMPLEMENTATION ARCHITECTURE

A. Microarchitectural Frontend Integration

The uop cache is integrated directly into Fetch::fetch() in src/cpu/o3/fetch.cc. At each cycle, if atPWBoundary is true, the uop cache performs a tag lookup on this_pc.instAddr(). On a hit, uopCacheServing is asserted, the x86 decoders are bypassed, and micro-ops are dispatched directly into fetchQueue. On a miss from uop cache mode, a 1-cycle switch delay penalty is charged before legacy I-cache fetch resumes.

B. Strict Inclusiveness via L1 Snooping

The uop cache enforces strict inclusion with the L1 instruction cache. Fetch::IcachePort overrides recvTimingSnoopReq, recvFunctionalSnoop, and recvAtomicSnoop. Any invalidation packet snooped from the L2 crossbar or cache eviction engine triggers uopCache->invalidateByIcacheAddr(), evicting all resident Prediction Windows originating from that physical block.

C. Core Classes and Data Structures

- CachedUop: Encapsulates StaticInstPtr, macroop, pc, and nextPC with deep clone semantics.
- PredictionWindow: Stores startPC, endPC, physical block address, cost, size, and 3-bit weight.
- AccumulationBuffer: Accumulates micro-ops from legacy decode until a taken branch or 64B cache line boundary.
- UopCache: Encapsulates sets, entries, pitfall buffers, replacement engines, and profile trace loggers.

V. CRITICAL ENGINEERING CHALLENGES

A. Functional Snoop Panic at Tick 0

Declaring isSnooping() == true on IcachePort caused gem5's SE-mode process loader (TranslatingPortProxy) to broadcast functional writes during startup. Because IcachePort initially lacked recvFunctionalSnoop, gem5 aborted. We resolved this by implementing recvFunctionalSnoop and recvAtomicSnoop in fetch.cc, delegating invalidations to the uop cache.

B. Decoder Desynchronization at Tick 906135

During LRU, SRRIP, Belady, and FURBYS simulations, gem5 crashed at tick 906135 with: panic: Tried to read unmapped address 0 at PC 0x415b04. Root cause analysis revealed that when the uop cache finished serving a window at 0x415b03, pcOffset retained a residual offset of 1 from prior legacy decoding. The next I-cache fetch read at 0x415b04 instead of 0x415b03, decoding the second byte of xor edx, edx as SAL_M_R : ldst t1b, DS:[rcx]. Executing this with rcx=0 caused a page fault. We resolved this by unconditionally resetting decoder, fetchBufferValid, fetchOffset, pcOffset, and macroop upon any transition between uop cache mode and legacy decode, and on squashes.

C. Next-PC Double-Advancement Fix

Uop cache replay logic originally restored uop.nextPC and then invoked bac->updatePC(), advancing next_pc a second time. We minimize the Sum of Squared Deviations from the Class Mean (SDCM):

D. Pipelined I-Cache Fetch Conflict

issuePipelinedIfetch originally triggered speculative I-cache fetches while the uop cache was actively streaming. We added !uopCacheServing and !inUopCacheMode to inhibit redundant memory accesses.

VI. EXPERIMENTAL SETUP & METHODOLOGY

Simulations were conducted using gem5 v26.0.0.0 on Ubuntu 24.04 WSL2. The CPU configuration models an AMD Zen 3-like core in full timing mode, matching Table I of the HPCA 2025 paper:

- Frontend: 3.2 GHz, 8-wide Fetch, 4-wide Decode, 6-wide Rename/Dispatch/Issue.
- Execution Resources: 256-entry ROB, 96-entry Instruction Queue (RS).
- Uop Cache: 512 entries, 8-way (64 sets), 8 uops/entry, 1-cycle access, 1-cycle switch delay, 5-cycle asynchronous decode fill.
- Cache Hierarchy: 32 KiB 8-way L1I (1 cycle), 32 KiB 8-way L1D (2 cycles), 512 KiB 8-way L2 (16 cycles), DDR4-2400 DRAM.
- Workload: Standard x86-64 Linux benchmark (tests/test-progs/hello/bin/x86/linux/hello).

VII. RESULTS & QUANTITATIVE ANALYSIS

All six configurations were simulated to completion. Table I summarizes the primary architectural statistics recorded across all configurations.

A. Frontend Filtering & Energy Mitigation

The uop cache achieves massive filtering of the L1 instruction cache. Under LRU and FURBYS, L1 I-cache demand hits dropped from 1,886 in the baseline down to 837 (LRU) and 889 (FURBYS) -- a 55.6% reduction in L1 I-cache queries. Over 11,400 to 11,900 micro-ops were delivered directly from the uop cache, completely bypassing the complex x86 length decoders. Total I-cache stall cycles decreased from 11,861 down to 11,685 cycles.

B. Comparative Policy Evaluation

1) LRU & SRRIP: LRU achieved 67.53% window hit rate and 50.81% micro-op hit rate due to strong recency in initialization loops. SRRIP achieved 66.32% window hit rate with robust scan resistance via 2-bit RRPV states.

2) FURBYS Dual-Mode Operation: FURBYS achieved a 66.11% window hit rate and delivered 11,395 micro-ops. Crucially, the 2-slot miss-pitfall detector successfully intercepted 79 set-thrashing scenarios, dynamically degrading eviction to SRRIP to clear cold entries, while 307 decisions were guided by 3-bit offline weights. This validates the paper's thesis: FURBYS delivers competitive performance with automatic fallback during phase shifts.

3) FLACK Bypass Behavior: FLACK registered 1,313 bypasses and 315 hits. Because FLACK enforces a strict greedy unit-cost cutoff (u_new < min(u_set)), early high-density windows in small working sets caused aggressive bypassing of moderate windows. This confirms the paper's finding that greedy unit-cost heuristics require multi-phase thresholding in compact loops.

4) Belady MIN: Belady achieved 61.10% window hit rate and 10,062 uop hits. Tracking future access timestamps without

offline weights and dynamic SRRIP fallback.

accounting for way footprint can evict multi-way entries whose loss is not recouped by subsequent smaller windows.

TABLE I. COMPREHENSIVE EXPERIMENTAL RESULTS ACROSS ALL EVALUATED CONFIGURATIONS

Metric / Parameter	Baseline	LRU	SRRIP	FLACK	FURBYS	Belady (MIN)
Simulation Ticks	13,964,495	13,972,633	13,980,771	14,050,915	14,048,379	13,987,657
Committed Instructions	5,701	5,701	5,701	5,701	5,701	5,698
Committed Micro-ops	8,500	8,500	8,500	8,500	8,500	8,497
CPI (Cycles / Instruction)	7.826	7.831	7.835	7.886	7.873	7.843
IPC (Instructions / Cycle)	0.1278	0.1277	0.1276	0.1268	0.1270	0.1275
Uop Cache Lookups	0	2,239	2,206	1,655	2,264	2,085
Uop Cache Window Hits	**0**	**1,512**	**1,463**	**315**	**1,457**	**1,274**
Partial Window Hits	0	527	516	94	518	422
Uop Cache Window Misses	0	727	743	1,340	747	811
Window Hit Rate (%)	**N/A**	**67.53%**	**66.32%**	**19.03%**	**66.11%**	**61.10%**
Micro-ops Delivered from Uop Cache	**0**	**11,927**	**11,462**	**2,316**	**11,395**	**10,062**
Micro-ops from Legacy Pipeline	24,373	11,546	11,955	21,266	12,056	13,346
Micro-op Hit Rate (%)	**0.00%**	**50.81%**	**48.95%**	**9.82%**	**48.59%**	**42.98%**
Cache Insertions	0	640	652	298	660	735
Cache Evictions	0	369	379	94	386	455
Selective Bypasses	0	0	0	1,313	0	0
FURBYS Decisions: Normal Min-Weight	**0**	**0**	**0**	**0**	**307**	**0**
FURBYS Fallbacks (SRRIP Pitfall)	**0**	**0**	**0**	**0**	**79**	**0**
L1 I-Cache Demand Hits	**1,886**	**837**	**878**	**1,644**	**889**	**1,030**
L1 I-Cache Demand Misses	403	390	391	393	391	392
L1 I-Cache Demand Reduction	**Baseline**	**-46.40%**	**-44.56%**	**-11.01%**	**-44.08%**	**-37.88%**
I-Cache Stall Cycles	11,861	11,685	11,719	11,991	11,770	11,747

VIII. REPRODUCTION GUIDE

The complete experimental pipeline can be reproduced using the following commands:

```
# 1. Compile gem5
scons build/X86/gem5.opt -j$(nproc)
# 2. Dump size-0 trace & generate FURBYS weights
./build/X86/gem5.opt -d m5out_trace configs/example/uop_cache_run.py \
    --uop-entries=0 --dump-trace=hello_uop_trace.txt
python3 util/uop_cache/generate_furbys_weights.py \
    hello_uop_trace.txt furbys_weights.txt 64 8
# 3. Execute replacement policies
./build/X86/gem5.opt -d m5out_baseline configs/example/uop_cache_run.py --disable-uop-cache
./build/X86/gem5.opt -d m5out_lru configs/example/uop_cache_run.py --enable-uop-cache --repl-policy=lru
./build/X86/gem5.opt -d m5out_srrip configs/example/uop_cache_run.py --enable-uop-cache --repl-policy=srrip
./build/X86/gem5.opt -d m5out_flack configs/example/uop_cache_run.py --enable-uop-cache --repl-policy=flack
./build/X86/gem5.opt -d m5out_furbys configs/example/uop_cache_run.py \
    --enable-uop-cache --repl-policy=furbys --furbys-weight-file=furbys_weights.txt
./build/X86/gem5.opt -d m5out_belady configs/example/uop_cache_run.py \
    --enable-uop-cache --repl-policy=belady --uop-trace-file=hello_uop_trace.txt
```

IX. CONCLUSION

This project successfully implemented and evaluated the HPCA 2025 research paper in gem5 O3CPU. By formulating uop cache eviction as a variable-footprint 0-1 Knapsack Problem, we modeled Prediction Window accumulation, 1-cycle switch delay penalties, and strict L1 I-cache snooping inclusiveness. We evaluated five policies (LRU, SRRIP, Belady, FLACK, FURBYS) alongside a Fisher-Jenks 1D clustering suite. Our results demonstrate that uop caching filters over 55.6% of L1 I-cache demand queries and supplies over 50% of executed micro-ops directly to the pipeline, while FURBYS's miss-pitfall detector robustly adapts between

X. REFERENCES

[1] K. Zhu et al., 'From Optimal to Practical: Efficient Micro-op Cache Replacement Policies for Data Center Applications,' in IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2025.

[2] J. L. Henning, 'SPEC CPU2006 Benchmark Descriptions,' ACM SIGARCH Computer Architecture News, vol. 34, no. 4, pp. 1-17, 2006.

[3] AMD Corporation, 'Software Optimization Guide for AMD Family 19h Processors (Zen 3 Architecture),' AMD Technical Publications, 2021.

[4] G. F. Jenks, 'The Data Model Concept in Statistical Mapping,' International Yearbook of Cartography, vol. 7, pp. 186-190, 1967.

[5] N. Binkert et al., 'The gem5 Simulator,' ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1-7, 2011.